

Lesson: Basic Types

CSE 4107 : Structured Programming I

Shahriar Ivan

Integer Types

Signed and Unsigned Integers

- Whole numbers
- Two types:
 - Signed Integer
 - Leftmost bit -> **Sign Bit**
 - Sign bit is 0 if the number is positive or zero, 1 if it's negative
 - 32 bits -> $[-2^{31}, 2^{31} - 1]$
 - Unsigned Integer
 - No sign bit
 - 32 bits -> $[0, 2^{32} - 1]$

Integer Types

Signed and Unsigned Integers

- By default, integer variables are signed in C—the leftmost bit is reserved for the sign
- To tell the compiler that a variable has no **sign** bit, declare it to be unsigned
- Unsigned numbers are primarily useful for systems programming and low-level, machine-dependent applications
- Use %o and %x with prefix for **unsigned** octal and hexadecimal, respectively

Integer Types

Integers in 32 bit computers

Data Type	Memory (Bytes)	Range	Format Specifier
short <code>int</code>	2	$[-2^{15}, 2^{15} - 1]$	%hd
unsigned short <code>int</code>	2	$[0, 2^{16} - 1]$	%hu
<code>int</code>	4	$[-2^{31}, 2^{31} - 1]$	%d
unsigned <code>int</code>	4	$[0, 2^{32} - 1]$	%u
long <code>int</code>	4	$[-2^{31}, 2^{31} - 1]$	%ld
unsigned long <code>int</code>	4	$[0, 2^{32} - 1]$	%lu
long long <code>int</code>	8	$[-2^{63}, 2^{63} - 1]$	%lld
unsigned long long <code>int</code>	8	$[0, 2^{64} - 1]$	%llu

Integer Types

Integer Constants

- Decimal (Base 10)

- $15 \rightarrow 1 \times 10^1 + 5 \times 10^0 = 15$
- Must not begin with zero in C. e.g.: 42, 22
- Compiler: int -> long int -> unsigned int

- Octal (Base 8)

- $237 \rightarrow 2 \times 8^2 + 3 \times 8^1 + 7 \times 8^0 = 159$
- Must begin with zero in C. e.g.: 017, 0377

- Hexadecimal (Base 16)

- $1AF \rightarrow 1 \times 16^2 + 10 \times 16^1 + 15 \times 16^0 = 431$
- Must begin with 0x in C. e.g.: 0xf, 0Xff, 0xFF, 0XFF, 0xfF

Integer Types

Integer Constants

- To force the compiler to treat a constant as a long integer, just follow it with the letter L (or l):

15L 0377L 0x7fffL

- To indicate that a constant is unsigned, put the letter U (or u) after it:

15U 0377U 0x7fffU

- L and U can be used in combination:

0xffffffffUL

- The order of the L and U doesn't matter, nor does their case

Integer Types

Integer Constants

- long long (C99) -> LL or ll (both cases must match)
- In C99, the compiler behavior
 - Decimal: int -> long -> long long
 - Octal or Hexadecimal: int -> unsigned int -> long -> unsigned long -> long long -> unsigned long long

Integer Types

Integer Constants

- For an *octal* or *hexadecimal* constant, the list of possible types is int, unsigned int, long int, unsigned long int, long long int, and unsigned long long int, in that order
- Any suffix at the end of a constant changes the list of possible types.
 - A constant that ends with U (or u) must have one of the types unsigned int, unsigned long int, or unsigned long long int
- A decimal constant that ends with L (or l) must have one of the types long int or long long int

Integer Types

Integer Overflow

- When arithmetic operations are performed on integers, it's possible that the result will be too large to represent
- For example, when an *arithmetic operation* is performed on two int values, the result must be able to be represented as an int
- If the result can't be represented as an int (because it requires too many bits), we say that **overflow** has occurred

Integer Types

Integer Overflow

- The behavior when integer overflow occurs depends on whether the operands were *signed* or *unsigned*
 - When overflow occurs during an operation on *signed* integers, the program's behavior is undefined
 - When overflow occurs during an operation on *unsigned* integers, the result *is* defined: we get the correct answer modulo 2^n , where n is the number of bits used to store the result

Programming Task

Summing a Series of Numbers (Revisited)

- During summation, the result might exceed the largest value allowed for an int variable

- Example: (For 64-bit computers)

```
Enter integers (0 to terminate):
```

```
9000000000 9000000000 0
```

```
The sum is: 820130816
```

- Improve the program using long long

Floating Types

- To store numbers with digits after decimal point -> Real numbers
- Type
 - float
 - Single-precision Floating-point
 - Precision isn't critical
 - double
 - Double-precision Floating-point
 - Greater precision - enough for most Programs
 - long double
 - Extended-precision Floating-point and rarely used format

Floating Types

- Most computers follow IEEE 754 standard which is developed by IEEE
- Two primary formats
 - Single Precision (32 bits)
 - Double Precision (64 bits)
- Numbers stored in a form of scientific notation
 - Sign
 - Exponent
 - Fraction
- Single-precision -> exponent is 8 bits long, fraction is 23 bits, 1 bit sign

Floating Types

- Characteristics of float and double when implemented according to the IEEE standard:

<i>Type</i>	<i>Smallest Positive Value</i>	<i>Largest Value</i>	<i>Precision</i>
float	1.17549×10^{-38}	3.40282×10^{38}	6 digits
double	2.22507×10^{-308}	1.79769×10^{308}	15 digits

- On computers that don't follow the IEEE standard, this table won't be valid
- In fact, on some machines, float may have the same set of values as double, or double may have the same values as long double

Floating Types

- Macros that define the characteristics of the floating types can be found in the `<float.h>` header.
- In C99, the floating types are divided into two categories.
 - ***Real floating types*** (float, double, long double)
 - ***Complex types*** (float _Complex, double _Complex, long double _Complex)

Floating Types

Floating Constants

- Floating constants can be written in a variety of ways
- Valid ways of writing the number 57.0:

57.0 57. 57.0e0 57E0 5.7e1 5.7e+1
.57e2 570.e-1

- A floating constant must contain a decimal point and/or an exponent; the exponent indicates the power of 10 by which the number is to be scaled
- If an exponent is present, it must be preceded by the letter E (or e). An optional + or - sign may appear after the E (or e)

Floating Types

Floating Constants

- By default, floating constants are stored as *double-precision* numbers
- To indicate that only single precision is desired, put the letter F (or f) at the end of the constant (for example, 57.0F)
- To indicate that a constant should be stored in long double format, put the letter L (or l) at the end (57.0L)

Floating Types

Reading and Writing Floating-Point Numbers

- Use %e, %f, or %g for single-precision
- For reading a value of type double, use l as prefix
- For writing a value of type double, %f is sufficient

```
double d;  
scanf("%lf", &d);  
printf("%f\n", d);
```

- For reading or writing a long double, put L as prefix

```
long double ld;  
scanf("%Lf", &ld);  
printf("%Lf\n", ld);
```

Character Types

Character Sets

- The only remaining basic type is char, the character type
- Today's most popular character set is **ASCII** (American Standard Code for Information Interchange), a **7-bit** code capable of representing **128** characters
- **ASCII** is often extended to a **256**-character code known as **Latin-1** that provides the characters necessary for Western European and many African languages

Character Types

Character Sets

- A variable of type char can be assigned any single character:

```
char ch;
```

```
ch = 'a'; /* lower-case a */
```

```
ch = 'A'; /* upper-case A */
```

```
ch = '0'; /* zero      */
```

```
ch = ' '; /* space     */
```

- Notice that character constants are enclosed in *single* quotes, not *double* quotes

Character Types

Operations on Characters

- C treats characters as small integers
 - 'a' -> 97
 - 'A' -> 65
 - '0' -> 48
 - '' -> 32

Character Types

Operations on Characters

- Character constants are int type -> C uses its integer value in computation

```
char ch;
```

```
int i;
```

```
i = 'a';      /* i is now 97  */
```

```
ch = 65;      /* ch is now 'A' */
```

```
ch = ch + 1;  /* ch is now 'B' */
```

```
ch++;        /* ch is now 'C' */
```

Character Types

Operations on Characters

- Characters can be compared, just as numbers can
- An if statement that converts a *lower-case* letter to *upper case*:

```
if ('a' <= ch && ch <= 'z')
```

```
    ch = ch - 'a' + 'A';
```

- Comparisons such as 'a' <= ch are done using the integer values of the characters involved
- These values depend on the character set in use, so programs that use <, <=, >, and >= to compare characters may not be portable

Character Types

Operations on Characters

- Characters have the same properties as numbers -> one of the **advantage**
- For example, we can write characters as control variables:

```
for (ch = 'A'; ch <= 'Z'; ch++) ...
```

- **Disadvantages** of treating characters as numbers: (**Solution** -> use **ctype.h**)
 - Can lead to errors that won't be caught by the compiler
 - Allows meaningless expressions such as 'a' * 'b' / 'c'
 - Can hamper portability

Character Types

Escape Sequences

- A character constant is usually one character enclosed in *single quotes*
- However, certain special characters—including the new-line character—can't be written in this way, because they're invisible (nonprinting) or because they can't be entered from the keyboard
- ***Escape sequences*** provide a way to represent these characters.
 - There are **two** kinds of escape sequences: ***character escapes*** and ***numeric escapes***

Character Types

Escape Sequences

- A complete list of character escapes:

<i>Name</i>	<i>Escape Sequence</i>
--------------------	-------------------------------

Alert (bell)	\a
--------------	----

Backspace	\b
-----------	----

Form feed	\f
-----------	----

New line	\n
----------	----

Carriage return	\r
-----------------	----

Horizontal tab	\t
----------------	----

<i>Name</i>	<i>Escape Sequence</i>
--------------------	-------------------------------

Vertical tab	\v
--------------	----

Backslash	\\
-----------	----

Question mark	\?
---------------	----

Single quote	\'
--------------	----

Double quote	\"
--------------	----

Character Types

Character Handling Functions

- Calling C's toupper library function is a fast and portable way to convert case:

```
ch = toupper(ch);
```

- toupper() returns the upper-case version of its argument.
- Programs that call toupper() need to have the following `#include` directive at the top:

```
#include <ctype.h> // This library function has other useful functions
```

Character Types

Reading and Writing Characters using scanf and printf

- The %c conversion specification allows scanf and printf to read and write single characters:

```
char ch;  
scanf("%c", &ch);  /* reads one character */  
printf("%c", ch);  /* writes one character */
```

- *scanf* doesn't skip white-space characters
- To force scanf to skip white space before reading a character, put a space in its format string just before %c:

```
scanf(" %c", &ch);
```

Character Types

Reading and Writing Characters using scanf and printf

- Since scanf doesn't normally skip white space, it's easy to detect the end of an input line
- A loop that reads and ignores all remaining characters in the current input line:

```
do {  
    scanf("%c", &ch);  
} while (ch != '\n');
```

- When scanf is called the next time, it will read the first character on the next input line

Character Types

Reading and Writing Characters using `getchar` and `putchar`

```
char ch;  
ch = getchar();    // reads a character, returns int  
putchar(ch);       // writes a character
```

- Saves execution time
- Consider the previous do-while loop:

```
do {  
    scanf("%c", &ch);  
} while (ch != '\n');
```

- Can be written as: `while((ch = getchar()) != '\n');`

Character Types

Reading and Writing Characters using `getchar` and `putchar`

- **Be careful** when *mixing* `getchar` and `scanf`
- `scanf` has a tendency to leave behind characters that it has “peeked” at but not read, including the new-line character:

```
printf("Enter an integer: ");  
scanf("%d", &i);  
printf("Enter a command: ");  
command = getchar();
```

- `scanf` will leave behind any characters that weren't consumed during the reading of `i`, including (but not limited to) the new-line character
- `getchar` will fetch the first leftover character

Programming Task

Determining the Length of a Message

- The program displays the *length* of a message entered by the user:

```
Enter a message: Brevity is the soul of wit.
```

```
Your message was 27 character(s) long.
```

- The length includes spaces and punctuation, but not the new-line character at the end of the message
- We could use either `scanf` or `getchar` to read characters; most C programmers would choose `getchar`

Type Conversion

- Required to make operands of the same size during arithmetic operation
- Generates instructions that change the **type** of some operands
so that the hardware will be able to evaluate the expression
- Example
 - If we add *16-bit* short and a *32-bit* int, short is converted to
32-bits
 - If we add an int and a float, int is converted to float format

Type Conversion

Implicit Conversion

- Compiler handles some conversions automatically -> Implicit conversions
- Implicit conversions are performed:
 - When the operands in an arithmetic or logical expression don't have the same type. (C performs what are known as the ***usual arithmetic conversions***)
 - When the type of the expression on the right side of an assignment doesn't match the type of the variable on the left side
 - When the type of an argument in a function call doesn't match the type of the corresponding parameter
 - When the type of the expression in a return statement doesn't match the function's return type

Type Conversion

The Usual Arithmetic Conversion

- Applied to the operands of most binary operators
- Consider the following

```
int i;
```

```
float f;
```

```
printf("%f", f+i);
```

- If `f` has type `float` and `i` has type `int`, the usual arithmetic conversions will be applied to the operands in the expression `f + i`

Type Conversion

The Usual Arithmetic Conversion

- Safer to convert `i` to `float` rather than convert `f` to `int`
 - `int` -> `float`
 - » Might cause minor loss of precision
 - `float` -> `int`
 - » Fractional part lost
 - » Meaningless if original number doesn't fit `int`'s range

Type Conversion

The Usual Arithmetic Conversion

- Strategy behind arithmetic conversions: convert operands to the “*narrowest*” type that will safely accommodate both values
- Operand types can often be made to match by converting the operand of the *narrower type* to the type of the other operand -> ***promotion***
- The rules for performing the usual arithmetic conversions can be divided into two cases:
 - The type of either operand is a floating type
 - Neither operand type is a floating type

Type Conversion

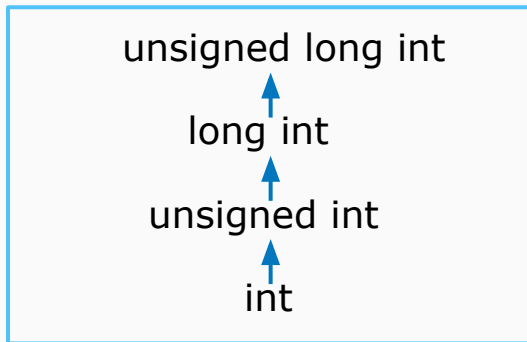
The Usual Arithmetic Conversion

- ***The type of either operand is a floating type***
 - If one operand has type long double, then convert the other operand to type long double
 - Otherwise, if one operand has type double, convert the other operand to type double
 - Otherwise, if one operand has type float, convert the other operand to type float

Type Conversion

The Usual Arithmetic Conversion

- ***Neither operand type is a floating type.*** First perform integral promotion on both operands
- Then use the following diagram to promote the operand whose type is narrower:



Type Conversion

The Usual Arithmetic Conversion

- When a signed operand is combined with an unsigned operand, the signed operand is converted to an unsigned value
- This rule can cause **obscure programming errors**
- It's best to use unsigned integers as little as possible and, especially, never mix them with signed integers

Type Conversion

The Usual Arithmetic Conversion

- Example of the usual arithmetic conversions:

```
char c;  
short int s;  
int i;  
unsigned int u;  
long int l;  
unsigned long int ul;  
float f;  
double d;  
long double ld;
```

```
i = i + c;    /* c is converted to int */  
i = i + s;    /* s is converted to int */  
u = u + i;    /* i is converted to unsigned int */  
l = l + u;    /* u is converted to long int */  
ul = ul + l;  /* l is converted to unsigned long int */  
f = f + ul;   /* ul is converted to float */  
d = d + f;    /* f is converted to double */  
ld = ld + d;  /* d is converted to long double */
```

Type Conversion

Conversion During Assignment

- Assigning a *floating-point* number to an integer variable drops the fractional part of the number:

```
int i;  
i = 842.97;      /* i is now 842 */  
i = -842.97;     /* i is now -842 */
```

- Assigning a value to a variable of a narrower type will give a meaningless result (or worse) if the value is outside the range of the variable's type:

```
c = 10000;        /*** WRONG ***/  
i = 1.0e20;       /*** WRONG ***/  
f = 1.0e100;      /*** WRONG ***/
```

Type Conversion

Implicit Conversions in C99

- Each integer type has an “integer conversion rank.”
- Ranks from highest to lowest:
 1. long long int, unsigned long long int
 2. long int, unsigned long int
 3. int, unsigned int
 4. short int, unsigned short int
 5. char, signed char, unsigned char
 6. _Bool
- Integer Promotion -> Involves converting any type less than int and unsigned int to int** or else to unsigned int

Type Conversion

Implicit Conversions in C99

- Type of either operand is a floating type
 - Same as before if neither is complex type
- Neither operand is a floating type
 - Perform integer promotion on both. Stop if both are same. If not:
 - If both are ***signed*** or both are ***unsigned*** then convert the lesser one to a greater rank
 - If ***unsigned*** has greater rank: convert the ***signed*** to ***unsigned*** type
 - If ***signed*** has greater rank: convert ***unsigned*** to ***signed***
 - If both are same ranked, convert ***signed*** to ***unsigned***
- All arithmetic type can be converted to `_Bool`: ***0*** -> *original value is 0*, ***1*** -> *otherwise*

Type Conversion

Casting

- Although C's implicit conversions are convenient, we sometimes need a greater degree of control over type conversion.
- For this reason, C provides **casts**.
- A cast expression has the form

(type-name) expression

type-name specifies the type to which the expression should be converted

Type Conversion

Casting

- Using a cast expression to compute the fractional part of a float value:

```
float f, frac_part;  
frac_part = f - (int) f;
```

- The difference between `f` and `(int) f` is the fractional part of `f`, which was dropped during the cast
- Cast expressions enable us to document type conversions that would take place anyway:

```
i = (int) f; /* f is converted to int */
```

Type Conversion

Casting

- Force the compiler to perform conversions:

```
float quotient;  
int dividend, divisor;  
scanf("%d %d", &dividend, &divisor);  
quotient = dividend / divisor;           // might cause problem  
quotient = (float)dividend / divisor;    // better way  
quotient = dividend / (float)divisor;    // also works  
quotient = (float)dividend / (float)divisor;
```

Type Conversion

Casting

- Casts are sometimes necessary to avoid overflow:

```
long i;  
int j = 1000;  
i = j * j;           /* overflow may occur */  
i = (long) j * j;    /* avoids overflow */
```

- The statement

```
i = (long) (j * j);  /*** WRONG ***/
```

wouldn't work, because overflow would already have occurred by the time of the cast

Type Definitions

- Create new types

```
typedef int Bool;
```

```
Bool flag;
```

- More understandable

```
typedef float Dollars;
```

```
Dollars debit, credit;
```

- Easy to modify :

```
typedef double Dollars;
```

- Portability from 32-bit to 16-bit :

```
typedef int Quantity;
```

The sizeof Operator

- The value of the expression

`sizeof ( type-name )`

is an unsigned integer representing the number of bytes required to store a value belonging to *type-name*

- `sizeof(char)` is always 1, but the sizes of the other types may vary
- On a *32-bit* machine, `sizeof(int)` is normally 4

The sizeof Operator

- The sizeof operator can also be applied to constants, variables, and expressions in general
 - If `i` and `j` are `int` variables, then `sizeof(i)` is 4 on a 32-bit machine, as is `sizeof(i + j)`
- When applied to an expression—as opposed to a type—sizeof doesn't require parentheses
 - We could write `sizeof i` instead of `sizeof(i)`
- Parentheses may be needed anyway because of operator precedence
 - The compiler interprets `sizeof i + j` as `(sizeof i) + j`, because `sizeof` takes precedence over binary `+`

The sizeof Operator

- Printing a sizeof value requires care, because the type of a sizeof expression is an *implementation-defined* type named `size_t`
- In C89, it's best to convert the value of the expression to a known type before printing it:

```
printf("Size of int: %lu\n",  
      (unsigned long) sizeof(int));
```

- The `printf` function in C99 can display a `size_t` value directly if the letter `z` is included in the conversion specification:

```
printf("Size of int: %zu\n", sizeof(int));
```